Sample Data Used for Examples

StudentID	StudentName	StudentAddress	StudentEnrolYear
01234567	John	285 Westport	2018
02345678	Anne	287 Eastport	2020
03456789	Robert	324 Northport	2019

tblStudent

CourseID	CourseName	CourseEnroll
ACT211	Financial Accounting	35
ACT212	Cost Accounting	28
MIS315	Information Systems	40
MIS322	Systems Analysis & Design	38
MIS432	Database Design	30
MKT311	Principles of Marketing	25
MGT490	Special Topics	20

tblCourse

StudentID	CourseID	Grade
01234567	ACT211	A+
01234567	ACT212	A
01234567	MIS315	B
02345678	ACT211	B+
02345678	MIS322	C
03456789	ACT212	B
03456789	MIS432	A
03456789	MKT311	A

tblGrading

Find the student's name and address of student ID 01234567 from the student table

```
SELECT StudentName, StudentAddress
FROM tblStudent
WHERE StudentID = '01234567';
```

StudentName	StudentAddress
John	285 Westport

Listing 6.6. SELECT to Select Specified Data from a Table

Find the student's entire record of student ID 01234567 from the student table."

-- The "*" sign represents all attributes of the table.

```
SELECT *
FROM tblStudent
WHERE StudentID = '01234567';
```

StudentID	StudentName	StudentAddress	StudentEnrolYear
01234567	John	285 Westport	2018

Listing 6.7. SELECT to Select an Entire Record from a Table

Find distinctive student enrollment years from the student table

```
SELECT DISTINCT StudentEnrolYear
  FROM tblStudent;
```

StudentEnrolYear
2018
2019
2020

Listing 6.8. DISTINCT to Eliminate Duplications

List the names of those students who enroll to the program after 2017

```
SELECT StudentName
FROM tblStudent
WHERE StudentEnrolYear > 2017;
```

StudentName
John
Anne
Robert

Listing 6.9. Comparison - Greater Than

List the names of those students whose ID numbers are greater than 00234567 and who enroll to the program after 2017 or before 2005

```
SELECT StudentName
FROM tblStudent
WHERE StudentID > '00234567'
AND (StudentEnrolYear > 2017 OR
StudentEnrolYear < 2005);
```

StudentName
John
Anne
Robert

Listing 6.10. AND and OR Operators

Find the student records for those students whose street names contain 'Westport'

-- The percent sign "%" is a wildcard to represent any
-- collection of characters

```
SELECT *
FROM tblStudent
WHERE StudentAddress LIKE '%Westport%';
```

StudentID	StudentName	StudentAddress	StudentEnrolYear
01234567	John	285 Westport	2018

Listing 6.11. LIKE Operator and Wildcard "%"

Find the student record for each student whose name has the letter 'o' as the second letter of the name

-- The wildcard sign "_" represents any one character.

```
SELECT *
FROM tblStudent
WHERE StudentName LIKE '_o%';
```

StudentID	StudentName	StudentAddress	StudentEnrolYear
01234567	John	285 Westport	2018
03456789	Robert	324 Northport	2019

Listing 6.12. LIKE Operator and Wildcard "_"

Find the student whose ID is ‘01234567’, ‘00234567’, or ‘00034567’

```
SELECT StudentName
FROM tblStudent
WHERE StudentID IN ('01234567',
'00234567', '00034567');
```

StudentName
John

Listing 6.13. IN Operator

SQL-User Interaction

- SQL is mainly embedded in host computer programming languages (C++, Java, .NET, COBOL...)
- SQL can be used on PC and includes user input request, depending on versions
- For example:

```
SELECT StudentName
FROM tblStudent
WHERE StudentID=[Please input student ID:];
```

List all student records in the reverse alphabetic order by student name

```
SELECT *
FROM tblStudent
ORDER BY StudentName DESC;
```

StudentID	StudentName	StudentAddress	StudentEnrolYear
03456789	Robert	324 Northport	2019
01234567	John	285 Westport	2018
02345678	Anne	287 Eastport	2020

Listing 6.14. ORDER BY Clause

Find the total student records in the student table

```
SELECT COUNT(*)
FROM tblStudent;
```

Expr1000
3

Listing 6.15(a). COUNT Function

Find the total student records in the student table

```
SELECT COUNT(*) AS CountOfStudents
FROM tblStudent;
```

CountOfStudents
3

Listing 6.15(b). Use AS Keyword to Define Result Name

What are the smallest enrollment number, largest enrollment number, total enrollment number, and average enrollment number in the course table?

```
SELECT MIN(CourseEnrollment),
       MAX(CourseEnrollment),
       SUM(CourseEnrollment),
       AVG(CourseEnrollment)
FROM tblCourse;
```

Expr1000	Expr1001	Expr1002	Expr1003
20	40	216	30.8571428571429

Listing 6.16. MIN, MAX, SUM, and AVG Functions

*Find the total number of courses taken by **each** student, and only include students who have taken at least 2 courses*

```
SELECT StudentID, COUNT(*)
FROM tblGrading
GROUP BY StudentID
HAVING COUNT(*) > 1;
```

StudentID	Expr1001
01234567	3
02345678	2
03456789	3

Listing 6.17. GROUP BY Clause and HAVING Clause

*Find the total number of courses taken by **each** student, and only include students whose ID are greater than '01234600' and have taken at least 2 courses*

```
SELECT StudentID, COUNT(*)
FROM tblGrading
WHERE StudentID>'01234600'
GROUP BY StudentID
HAVING COUNT(*)>1;
```

StudentID	Expr1001
02345678	2
03456789	3

Listing 6.18. Apply WHERE Clause before GROUP BY Clause

Arithmetic Operations

- The major purpose of SQL is data retrieval from the database together with simple data manipulations
- Complicated manipulations on the retrieved data and information presentations shall be implemented by the application programs that host SQL scripts
- Nevertheless, SQL provides **arithmetic operations**. Addition (+), subtraction (-), multiplication (*), and division (/) can be used in the SELECT clause

Arithmetic Operations (cont'd)

List the course ID, course enrollment, and seats available for each course, assuming that the normal cap for each course is 30

```
SELECT CourseID, CourseEnrollment,
30-CourseEnrollment AS SeatsAvailable
FROM tblCourse;
```

CourseID	CourseEnrollment	SeatsAvailable
ACT211	35	-5
ACT212	28	2
MGT490	20	10
MIS315	40	-10
MIS322	38	-8
MIS432	30	0
MKT311	25	5

Listing 6.19. Arithmetic Operations

Join – Merge Tables

- (1) A condition associates two tables, and the general format is:

  [*TableOnOneSide*].[*PrimaryKey*] = [*TableOnManySide*].[*ForeignKey*]

- (2) If *n* tables are used in the query, then *n*-1 conditions are needed and are tied by the AND operator

- (3) To differentiate the same names in different tables, the table name followed by a period sign is used for an attribute name (e.g., tblStudent.StudentID) to **qualify** the attribute name. In other words, the table name must be quoted to specify what table the attribute belongs to

List the names of those students who receive "A" or "A+" in any course

```
SELECT DISTINCT tblStudent.StudentName
FROM tblGrading, tblStudent
WHERE
tblStudent.StudentID=tblGrading.StudentID
AND (tblGrading.Grade='A+' OR
     tblGrading.Grade='A');
```

StudentName
John
Robert

Listing 6.20. Join Two Tables

```
SELECT  *
FROM tblStudent, tblCourse, tblGrading
WHERE tblStudent.StudentID = tblGrading.StudentID
AND tblCourse.CourseID = tblGrading.CourseID;
```

tblStudent.St	StudentNar	StudentAddre	StudentEnr	tblCourse.(	CourseName	CourseEnr	tblGrading.	tblGrading	Grade
01234567	John	285 Westport	2018	ACT211	Financial Accou	35	01234567	ACT211	A+
01234567	John	285 Westport	2018	MIS315	Information Sys	40	01234567	MIS315	B
02345678	Anne	287 Eastport	2020	ACT211	Financial Accou	35	02345678	ACT211	B+
02345678	Anne	287 Eastport	2020	MIS322	Systems Analysi	38	02345678	MIS322	C
03456789	Robert	324 Northport	2019	MIS432	Database Desig	30	03456789	MIS432	A
03456789	Robert	324 Northport	2019	ACT212	Cost Accounting	28	03456789	ACT212	B
03456789	Robert	324 Northport	2019	MKT311	Principles of Ma	25	03456789	MKT311	A
01234567	John	285 Westport	2018	ACT212	Cost Accounting	28	01234567	ACT212	A

Listing 6.21. Join Multiple Tables to Integrate Related Data

Who (student ID and name) receives "A+" or "A" grade in which course (course ID and course name)? List the results in order of student ID

```
SELECT tblGrading.StudentID, tblStudent.StudentName,
tblGrading.CourseID,tblCourse.CourseName,
tblGrading.Grade
FROM tblGrading, tblStudent, tblCourse
WHERE tblStudent.StudentID=tblGrading.StudentID
AND tblCourse.CourseID=tblGrading.CourseID
AND (tblGrading.Grade='A+' OR tblGrading.Grade='A')
ORDER BY tblStudent.StudentID;
```

StudentID	StudentName	CourseID	CourseName	Grade
01234567	John	ACT212	Cost Accounting	A
01234567	John	ACT211	Financial Accounting	A+
03456789	Robert	MKT311	Principles of Marketing	A
03456789	Robert	MIS432	Database Design	A

Listing 6.22. Query with Multiple Table

```
SELECT DISTINCT tblStudent.StudentName
FROM tblStudent
INNER JOIN tblGrading
ON
tblStudent.StudentID=tblGrading.StudentID
WHERE (tblGrading.Grade='A+' OR
       tblGrading.Grade='A');
```

Listing 6.23. Explicit Format of Inner Join

This format is not recommended for beginners

Subquery

(1) Reduce computational workload of join operation

- Avoid join operation

```
SELECT tblStudent.StudentName
FROM   tblStudent
WHERE  tblStudent.StudentID IN
          (SELECT tblGrading.StudentID
           FROM tblGrading
           WHERE(tblGrading.Grade='A+'
          OR tblGrading.Grade='A'));
```

StudentName
John
Robert

Listing 6.25. An Example of Subquery that Avoids Join Operation

Subquery

- Make the joining tables smaller

List all student grade records, including only the student numbers, the student names, the course numbers, and the grades

```
SELECT tblStudent.StudentID, tblStudent.StudentName,
     tblCourse.CourseID, tblGrading.Grade
FROM
(SELECT tblStudent.StudentID, tblStudent.StudentName
  FROM tblStudent) tblStudentSmall,
(SELECT tblCourse.CourseID
  FROM tblCourse) tblCourseSmall,
 tblGrading
WHERE tblStudentSmall.StudentID=tblGrading.StudentID
AND tblCourseSmall.CourseID=tblGrading.CourseID;
```

Listing 6.26. An Example of Subquery in the FROM Clause

StudentID	StudentName	CourseID	Grade
01234567	John	ACT211	A+
01234567	John	MIS315	B
02345678	Anne	ACT211	B+
02345678	Anne	MIS322	C
03456789	Robert	MIS432	A
03456789	Robert	ACT212	B
03456789	Robert	MKT311	A
01234567	John	ACT212	A

Subquery

(2) An alternative to the GROUP BY clause

Show each student's name along with the number of courses she/he has taken

```
SELECT tblStudent.StudentName,
    (SELECT COUNT(*)
     FROM tblGrading
     WHERE
     tblStudent.StudentID=tblGrading.StudentID)
     AS NumberOfCourses
FROM tblStudent;
```

StudentName	NumberOfCourses
John	3
Anne	2
Robert	3

Listing 6.27(a). Subquery for Groups

GROUP BY seems to be easier

The SQL in Listing 6.27(b) does the similar work using GROUP BY clause

```
SELECT tblStudent.StudentName,
       COUNT(*) AS NumberOfCourses
FROM tblStudent, tblGrading
WHERE
tblStudent.StudentID=tblGrading.StudentID
GROUP BY tblStudent.StudentName;
```

Listing 6.27(b). GROUP BY for Group

Subquery

(3) A subquery represents a variable

Find the percentage of the enrollment of MIS432 out of the total enrollment of all courses

```
SELECT SUM(CourseEnrollment)/
 (SELECT SUM(CourseEnrollment) FROM tblCourse)*100
  AS MIS432EnrollmentPercent
FROM   tblCourse
WHERE  CourseID = 'MIS432';
```

MIS432EnrollmentPercent
13.8888888888889

Listing 6.28. An Example of Subquery as a Variable

Subquery

(4) Determine uncertain criteria

Which students with ID numbers greater than '02000000' have the earliest enrollment year of <u>such</u> students?

```
SELECT StudentName
FROM   tblStudent
WHERE  StudentID > '02000000'
AND StudentEnrolYear=
      (SELECT MIN(StudentEnrolYear)
       FROM tblStudent
       WHERE  StudentID > '02000000');
```

StudentName
Robert

Listing 6.29. An Example of Subquery for Uncertain Condition

List student names, course names, and grades of those students whose ID numbers are greater than '02000000' and have the earliest enrollment year of such students

```
SELECT tblStudent.StudentName,
  tblCourse.CourseName,        tblGrading.Grade
FROM tblStudent, tblCourse, tblGrading
WHERE tblStudent.StudentID=tblGrading.StudentID
AND tblCourse.CourseID=tblGrading.CourseID
AND tblStudent.StudentID > '02000000'
AND tblStudent.StudentEnrolYear=
  (SELECT MIN(tblStudent.StudentEnrolYear)
  FROM tblStudent
  WHERE  tblStudent.StudentID > '02000000');
```

StudentName	CourseName	Grade
Robert	Database Design	A
Robert	Cost Accounting	B
Robert	Principles of Marketing	A

Listing 6.30. An Example of Join and Subquery

Two important points of subquery for uncertain condition

- First, when the right side of the WHERE clause is uncertain (e.g., MIN, MAX, SUM, AVG, COUNT), you must use a subquery to replace the uncertain condition
- Second, in Listing 6.29, you can see that the condition StudentID>'02000000' in the host WHERE clause repeats in the condition in the subquery. If the second WEHRE clause is omitted, the query is quite different from the original meaning, and stands for "*which students with ID numbers greater than '02000000' have the earliest enrollment year of <u>all</u> students?*"

UNION Operator

List the course ID and course enrollment for each course that has enrollment greater than 35, or the student name and student ID for each student who enrolled after 2019

```
SELECT CourseID AS Name, CourseEnrollment AS Data
FROM tblCourse
WHERE CourseEnrollment>35
UNION
SELECT StudentName AS Name, StudentID AS Data
FROM tblStudent
WHERE StudentEnrolYear>2019;
```

Name	Data
Anne	02345678
MIS315	40
MIS322	38

Listing 6.31. UNION Operator

(This example is merely to demonstrate how the UNION operator works. The query result is a kind of "orange" and "apple".)

Tactics for Writing Queries

- (1) Read the query carefully. Determine what data are to be retrieved and what attributes are to be included in the SELECT command
- (2) If a variable is needed, use a subquery as the variable
- (3) Determine what tables are will be used in the FROM clause
- (4) If two or more tables are involved, use a join operation(s) (match the primary key in one table with the foreign key in another table) in the WHERE clause

Tactics for Writing Queries (cont'd)

- (5) Construct the WHERE clause by including all conditions that are linked by AND or OR. Never use any aggregate function in the WHERE clause directly
- (6) If a condition has an uncertain criterion (MAX, MIN, AVG, SUM, COUNT) on the right side of the condition, use a subquery
- (7) Consider GROUP BY (for each group) with HAVING condition, ORDER BY clauses, and other operators (e.g., DISTINCT) if needed
- (8) For hands-on practices, you may construct test tables with a limited number of test samples to test the SQL script to see if it generates the expected result